Maîtriser le langage procédural sous MySQL

Introduction générale

Définition :

Un SGBD relationnel (Système de Gestion de Base de Données) est un logiciel qui permet de **stocker, organiser et manipuler les données de manière structurée**.

Exemple :

Une boutique en ligne peut avoir une base eshop_app_db contenant des tables : clients, produits, commandes.

Analogie:

Concept	Analogie bibliothèque
Table	Étageres contenant des fiches
Ligne	Une fiche client sur l'étagère
Colonne	Informations sur le client (nom, âge…)
SQL	Langage pour lire, ajouter ou modifier les fiches

Rappel SQL – Langage de manipulation des données

Création et suppression de base

Formateur : AARAB HAFID

-- Créer une base de données

Requête: CREATE DATABASE eshop_app_db;

CREATE DATABASE : commande SQL pour créer une base.

eshop_app_db : nom de la base.

Nb : Créer seulement si elle n'existe pas :

Requête: CREATE DATABASE IF NOT EXISTS eshop_app_db;

-- Supprimer une base de données

Requête: DROP DATABASE eshop_app_db;

- *DROP DATABASE : supprime la base et toutes ses tables définitivement.*

Erreur fréquente : Les données sont perdues définitivement.

Création et modification de table

-- Sélectionner la base sur laquelle travailler

Requête: USE eshop_app_db;

- *USE : Indique à MySQL sur quelle base exécuter les commandes suivantes.*

-- Créer une table clients

Requête: CREATE TABLE clients (

id INT NOT NULL AUTO_INCREMENT, *-- Identifiant unique, incrément automatique*

nom CHAR(50) NOT NULL, *-- Nom obligatoire, max 50 caractères*

prenom CHAR(40), *-- Prénom facultatif, max 40 caractères*

date_naissance DATE, *-- Date de naissance*

adresse VARCHAR(50) DEFAULT '', *-- Adresse, vide par défaut*

PRIMARY KEY(id) *-- Clé primaire pour identifier chaque ligne*

) ENGINE=InnoDB; *-- Moteur de stockage recommandé*

- *AUTO_INCREMENT : MySQL crée un ID unique automatiquement.*
- *PRIMARY KEY(id) : empêche les doublons et identifie chaque ligne.*
- *ENGINE=InnoDB : moteur moderne avec intégrité référentielle.*

Modifier une table :

Requête: ALTER TABLE clients ADD email VARCHAR(255); *-- Ajouter colonne email*

Requête: ALTER TABLE clients DROP COLUMN pays; *-- Supprimer colonne pays*

Requête: ALTER TABLE clients MODIFY COLUMN date_naissance DATE; *-- Changer type colonne*

Requête: ALTER TABLE clients RENAME TO acheteurs; *-- Renommer table*

Nb : Toujours vérifier avec : **Requête**: DESCRIBE clients; *-- afficher la structure de la table nommée "clients"*

Insertion, mise à jour et suppression

-- Ajouter un client

Requête: INSERT INTO clients(nom, prenom, date_naissance, adresse)

VALUES('Alami','Said','1980-05-22','Casablanca');

- *INSERT INTO clients(...) : spécifie les colonnes à remplir.*
- *VALUES(...) : les valeurs correspondantes pour la nouvelle ligne.*

-- Mettre à jour un client

Requête: UPDATE clients

SET nom='Fadani', adresse='Rabat'

WHERE id=1;

- *UPDATE clients : table ciblée.*
- *SET nom='Fadani', adresse='Rabat' : nouvelles valeurs.*
- *WHERE id=1 : limite la modification à l'ID 1.*

-- Supprimer un client

Requête: DELETE FROM clients

WHERE id=3;

- *DELETE FROM clients : supprime des lignes.*
- *WHERE : très important pour éviter de tout supprimer.*

Nb : Vérifier toujours avec : **Requête:** SELECT * FROM clients;

Exemple simulation :

id	nom	prenom	date_naissance	adresse
1	Alami	Said	1980-05-22	Casablanca
2	Zaki	Fatima	1986-05-22	Agadir
3	Fadani	Ali	1990-08-10	Casablanca

Erreur fréquente : Oublier le WHERE dans UPDATE ou DELETE.

Sélection des données

-- Sélectionner tous les clients de Casablanca, triés par nom

Requête: SELECT nom, prenom, date_naissance

FROM clients

WHERE adresse='Casablanca'

ORDER BY nom ASC;

- *SELECT nom, prenom, date_naissance : colonnes à afficher.*
- *FROM clients : table ciblée.*
- *WHERE adresse='Casablanca' : filtre sur la colonne adresse.*
- *ORDER BY nom ASC : tri croissant par nom.*

Résultat simulé :

nom	prenom	date_naissance
Alami	Said	1980-05-22
Fadani	Ali	1990-08-10

Vues et tables temporaires

Vues :

-- Créer une vue pour les clients d'Agadir

Requête: CREATE VIEW vw_clients_agadir AS

SELECT * FROM clients

WHERE UPPER(adresse) = 'AGADIR';

- *CREATE VIEW vw_clients_agadir : crée une table virtuelle.*
- *SELECT * FROM clients ... : définit le contenu de la vue.*
- *UPPER() : rend la comparaison insensible à la casse.*

Tables temporaires :

-- Créer une table temporaire

Requête: CREATE TEMPORARY TABLE temp_clients

SELECT * FROM clients LIMIT 0;

-- Supprimer la table temporaire

Requête: DROP TEMPORARY TABLE temp_clients;

- *TEMPORARY : table accessible uniquement pendant la session.*
- *LIMIT 0 : crée la structure sans données.*

Langage procédural MySQL (PSM : Persistent Stored Module)

Définition :

Permet de combiner SQL avec des structures de contrôle :

- **Conditionnelles :** IF, CASE
- **Boucles :** WHILE, REPEAT, LOOP
- **Directives :** LEAVE (sortir d'une boucle), ITERATE (recommencer un tour)

Analogie :

- SQL = lire/écrire les données
- PSM = automatiser les actions sur ces données

Types de programmes MySQL – Procédures, Fonctions et Triggers

Procédures stockées

Définition :

Une **procédure stockée** est un ensemble de requêtes SQL regroupées et stockées dans la base de données, que l'on peut exécuter à tout moment.

- Elle peut avoir des paramètres d'entrée et de sortie.

- Utile pour automatiser des tâches répétitives (exemple : vérifier stock, enregistrer ventes, calculer taxes…).

Exemple :

Requête:

```
DELIMITER $$  -- Changer le séparateur pour inclure ';' dans le corps
CREATE PROCEDURE sp_client_par_ville(IN ville VARCHAR(50))
BEGIN
   -- Sélectionner tous les clients correspondant à la ville
   SELECT *
   FROM clients
   WHERE UPPER(adresse) = UPPER(ville);
END$$
DELIMITER ;  -- Revenir au séparateur par défaut
```

1. *DELIMITER $$: permet d'utiliser le point-virgule ; à l'intérieur de la procédure sans terminer la commande.*
2. *CREATE PROCEDURE sp_client_par_ville : nom de la procédure.*
3. *(IN ville VARCHAR(50)) : paramètre d'entrée nommé ville.*
4. *BEGIN ... END : bloc contenant toutes les instructions SQL de la procédure.*
5. *SELECT * FROM clients WHERE UPPER(adresse) = UPPER(ville); : requête qui retourne tous les clients de la ville (insensible à la casse).*
6. *DELIMITER ; : revient au séparateur par défaut.*

Appel de la procédure :

Requête: CALL sp_client_par_ville('Agadir');

Résultat simulé :

id	nom	prenom	date_naissance	adresse
2	Zaki	Fatima	1986-05-22	Agadir

Fonctions

Définition :

Une **fonction MySQL** est un bloc de code qui retourne **une valeur unique** et peut être réutilisée dans plusieurs requêtes SQL.

- Elle peut avoir des paramètres d'entrée.
- Ne peut pas renvoyer une table complète, seulement une valeur (nombre, texte, date…).

Exemple :

Requête:

```
DELIMITER $$
CREATE FUNCTION uf_nombre_clients_par_ville(ville VARCHAR(50))
RETURNS INT
READS SQL DATA
BEGIN
   DECLARE nombre INT;  -- Déclare une variable locale
   SET nombre = (SELECT COUNT(*) FROM clients WHERE adresse = ville);  -- Compter clients
   RETURN nombre;  -- Retourner le résultat
END$$
DELIMITER ;
```

1. *CREATE FUNCTION : crée une fonction dans la base.*
2. *uf_nombre_clients_par_ville : nom de la fonction.*
3. *(ville VARCHAR(50)) : paramètre d'entrée.*
4. *RETURNS INT : la fonction retourne un entier.*
5. *READS SQL DATA : la fonction lit les données mais ne les modifie pas.*
6. *DECLARE nombre INT; : variable locale pour stocker le résultat.*
7. *SET nombre = (SELECT COUNT(*)...) : on affecte à nombre le nombre de clients dans la ville.*
8. *RETURN nombre; : renvoie la valeur calculée.*

Appel de la fonction :

Requête: SELECT uf_nombre_clients_par_ville('Casablanca') AS nb_clients;

Résultat simulé :

nb_clients

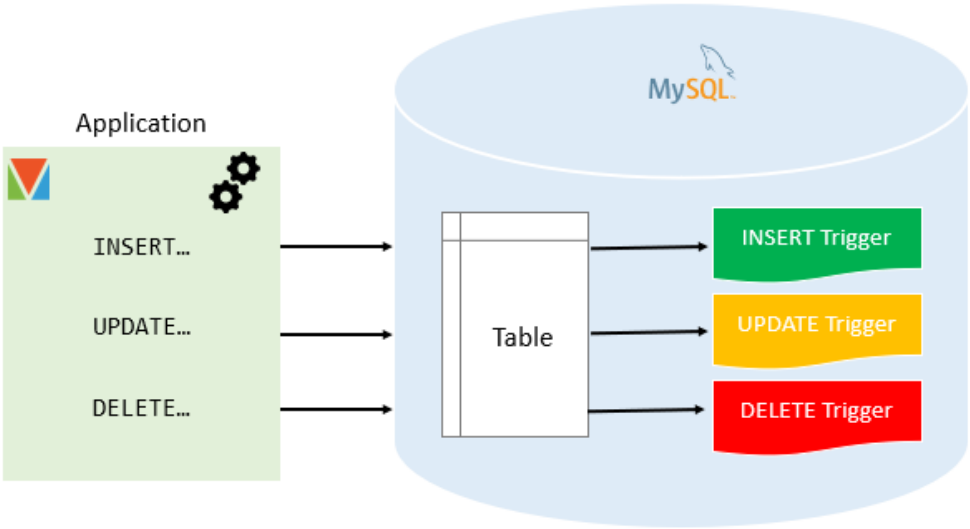
2

Nb : Utiliser une procédure si vous voulez renvoyer un ensemble de lignes (table complète).

Déclencheurs (Triggers)

Définition :

Un **trigger** est une action automatique qui se déclenche **avant ou après** un INSERT, UPDATE ou DELETE sur une table.



- Sert à automatiser la mise à jour de données, journaux ou vérifications.
- S'exécute **sans que l'utilisateur n'ait à appeler explicitement** la commande.

Exemple :

Requête:

```
CREATE TRIGGER
maj_date_modif

BEFORE UPDATE ON clients
-- Avant mise à jour de chaque
ligne

FOR EACH ROW                    --
Pour chaque ligne affectée

SET NEW.date_modification =
NOW(); -- Mettre
automatiquement la date
```

1. *CREATE TRIGGER maj_date_modif : nom du trigger.*
2. *BEFORE UPDATE : le trigger se déclenche avant la modification.*
3. *ON clients : table ciblée.*
4. *FOR EACH ROW : déclenchement pour chaque ligne affectée.*
5. *SET NEW.date_modification = NOW(); : met à jour la colonne date_modification avec la date actuelle.*

Nb : Toujours documenter les triggers car ils ne sont pas visibles dans les SELECT.

OFPPT

Formateur : AARAB HAFID